

Software Engineering

Lecture 05 Project Management

Prof. Dr. Ir. Riri Fitri Sari MM MSc

Faculty of Engineering
University of Indonesia

Software Engineering Roadmap: Chapter 2 Focus

**Corporate
practices**

**Plan
project**

**Analyze
requirements**

Design

Implement

Test units

**Integrate
& test system**

Maintain

Software
Engineering
Roadmap:
Chapter 2 Focus

Corporate
practices

**Plan
project**

Analyze
requirements

Design

Implement

Test units

Integrate
& test system

Maintain

*Development
phases*

**Management
structure**
- hierarchical, peer,...

**Development
process**
- when to do what phase
- document: SPMP

**Risk identification
& retirement**

SPMP

Schedule

Cost estimate

- Understand the term “project management”
- Organize teams
- Specify project management plans
- Define and retire risks
- Estimate costs very early in the life cycle
- Create high level projects schedules
- Write a Software Project Management Plan

1. Introduction to project management

Can somewhat vary the following factors.

1. The total cost of the project,
 - e.g., increase expenditures
2. The capabilities of the product,
 - e.g., subtract from a list of features
- • • • •

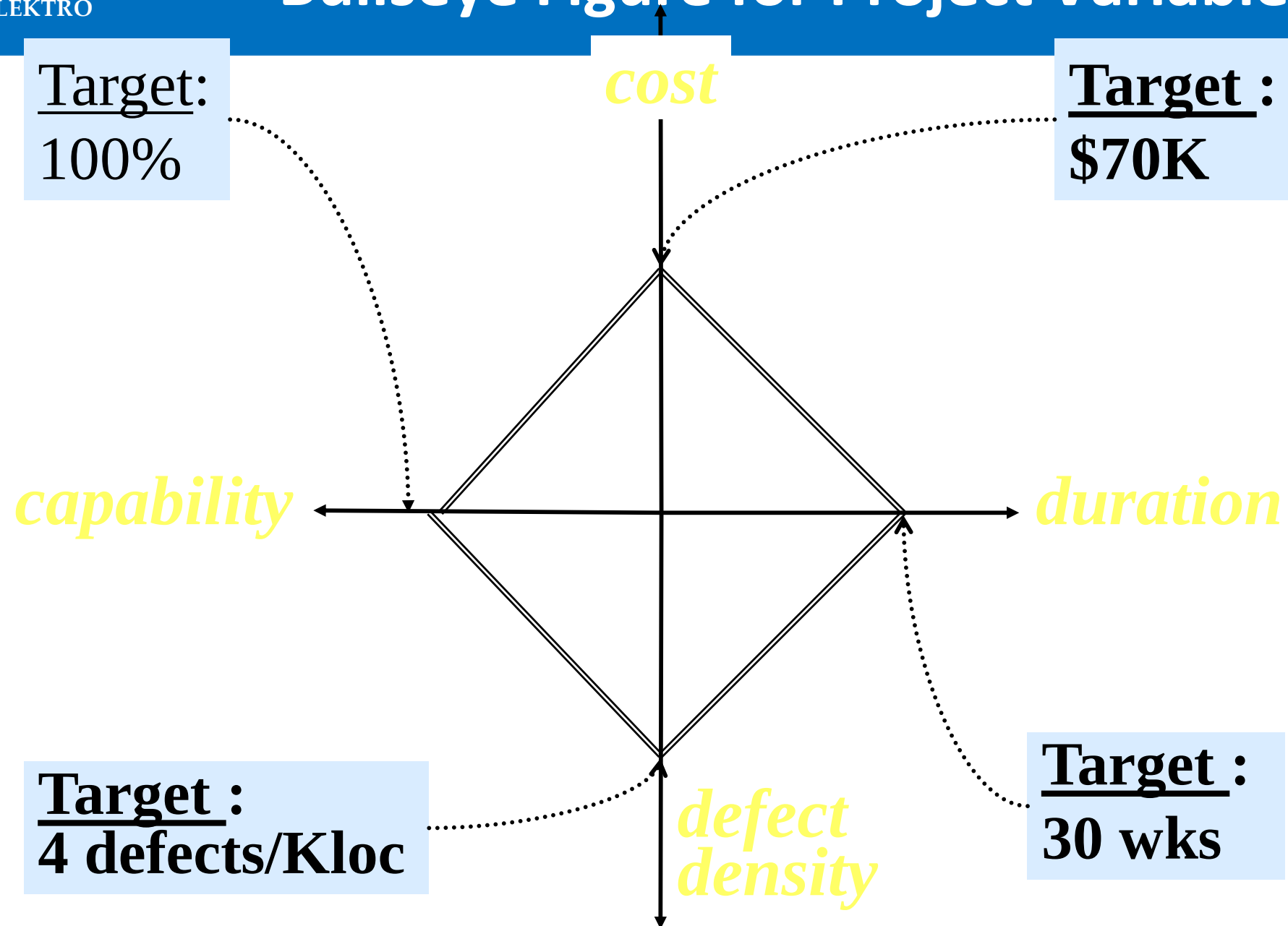
The Variables of Project Management

Can somewhat vary the following factors.

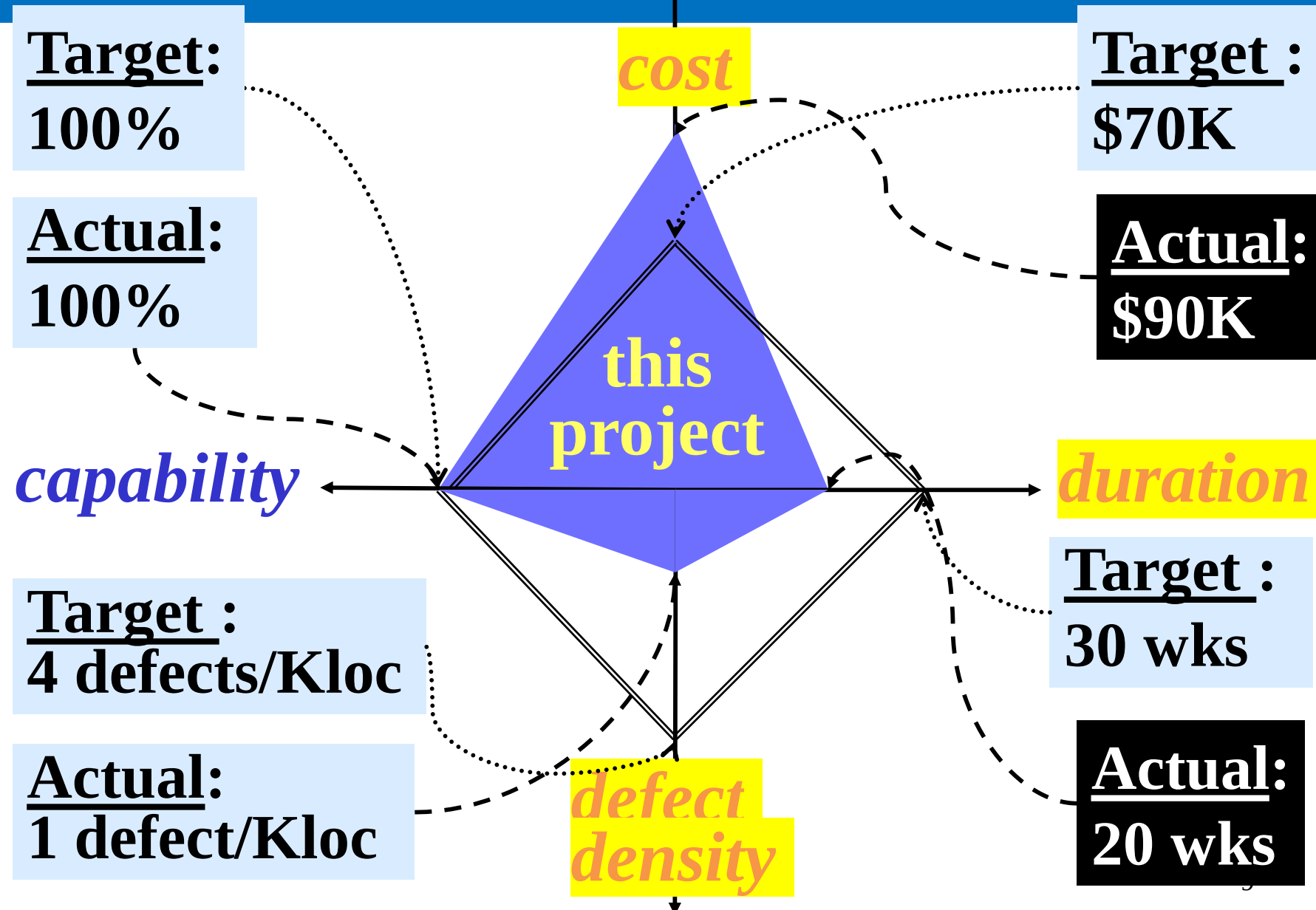
1. The total cost of the project,
 - e.g., increase expenditures
2. The capabilities of the product,
 - e.g., subtract from a list of features
3. The quality of the product,
 - e.g., increase the mean time between failure
4. The date on which the job is completed.
 - e.g., reduce the schedule by 20%
 - e.g., postpone project's completion date one month

The Variables of Project Management

Bullseye Figure for Project Variables



Bullseye Figure for Project Variables



1. Understand project content, scope, & time frame

2. Identify development process
(methods, tools, languages, documentation and support) -- section 4 of chapter 1

3. Determine organizational structure
(organizational elements involved) -- see section 3

4. Identify managerial process
(responsibilities of the participants) -- see section 3 of case study 1 at end of chapter

...

1. Understand project content, scope, & time frame

2. Identify development process

(methods, tools, languages, documentation and support) -- section 4 of chapter 1

3. Determine organizational structure

(organizational elements involved) -- see section 3

4. Identify managerial process

(responsibilities of the participants) -- see section 3 of case study 1 at end of chapter

5. Develop schedule

(times at which the work portions are to be performed) -- see section 6

6. Develop staffing plan -- see section 3.5 of case study 1

7. Begin risk management -- see section 4

8. Identify documents to be produced -- see SQAP 4.2

9. Begin process itself -- described in chapters 3-10

2. Managing people

- 1.* Distribute start time, end time, and agenda with approximate times (see figure tbd)
 - list important items first
- 2.* Ensure “strawman” items prepared
3. Start on time
4. Have someone record action items[‡]
5. Have someone track time & prompt members
6. Get agreement on the agenda and timing
7. Watch timing throughout, and end on time
 - allow exceptions for important discussion
 - stop excessive discussion; take off line
8. Keep discussion on the subject
- 9.** E-mail action items & decision summary.



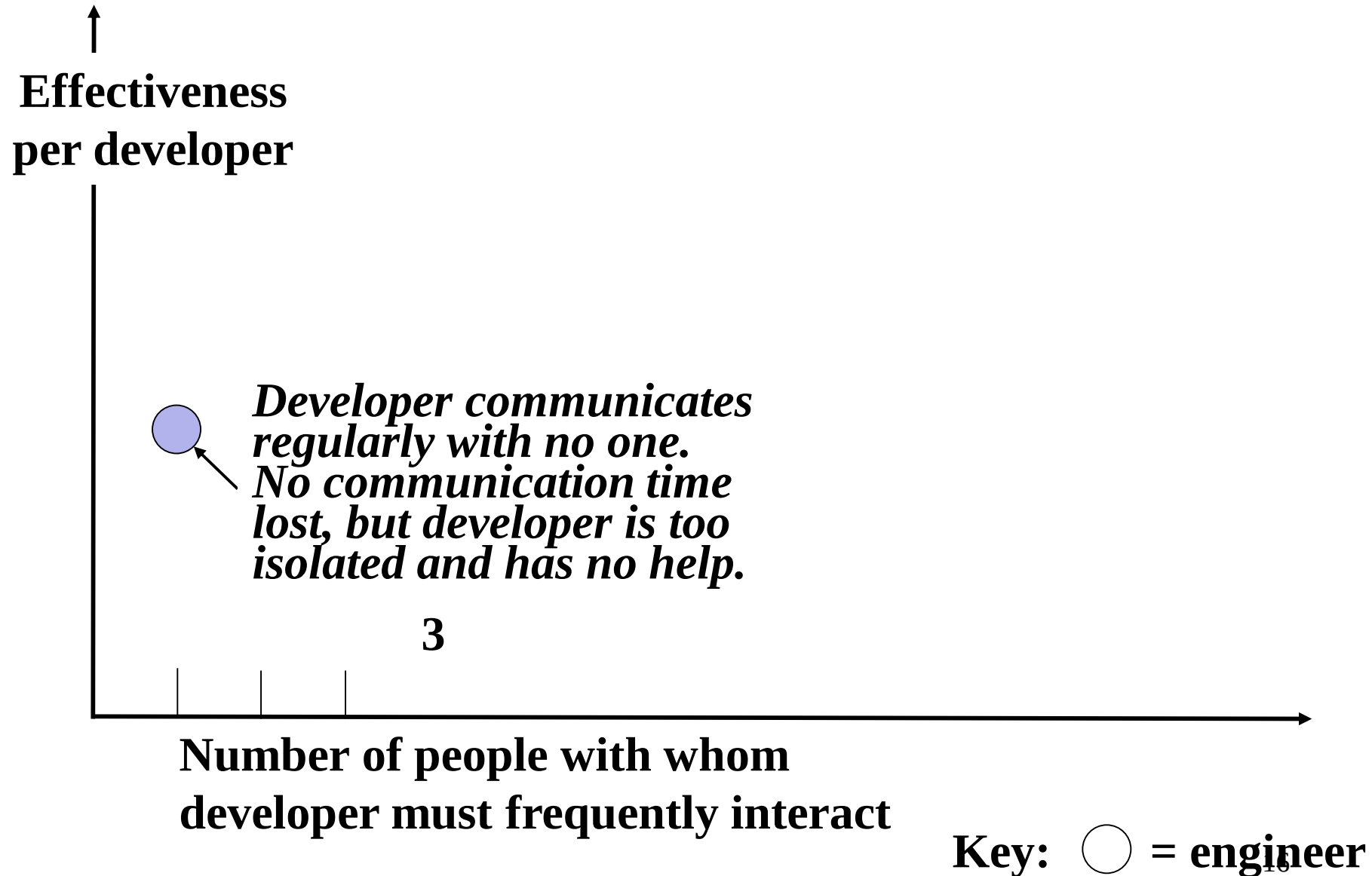
*** in advance of meeting ‡ actions members must perform ** after meeting**

Adapted from *Software Engineering: An Object-Oriented Perspective* by Eric J. Braude (Wiley 2001), with permission.

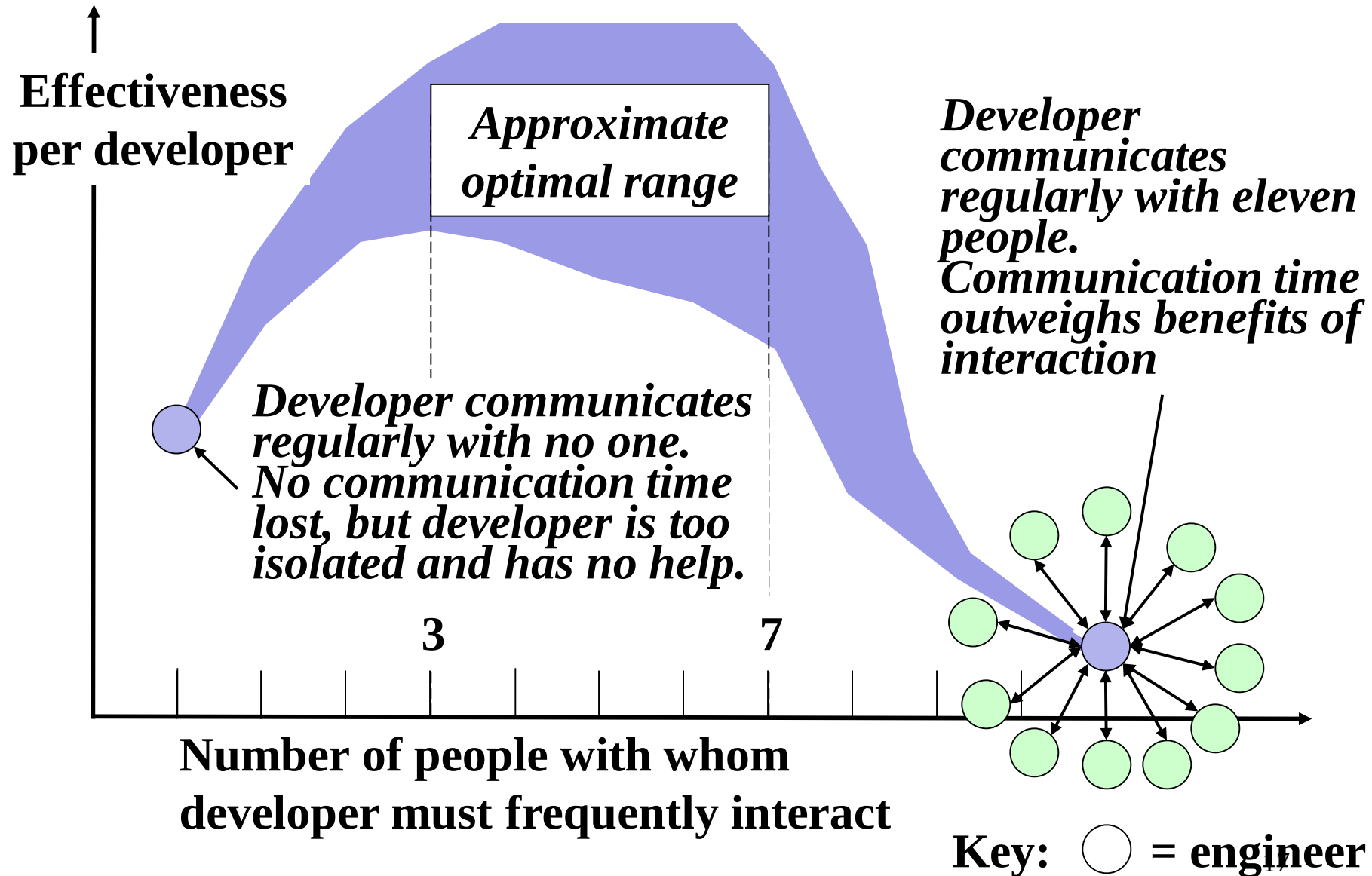
1. Get agreement on agenda & time allocation
2. Get volunteers to ... :
 - ... record decisions taken and action items
 - ... watch time and prompt members (see figure tbd)
3. Report progress on project schedule -- *10 mins*
4. Discuss strawman artifact(s) -- *x mins*
5. Discuss risk retirement -- *10 mins*
 - <MORE ITEMS>
 - metrics and process improvement?
- n. Review action items -- *5 mins*

3. Options for organizing personnel

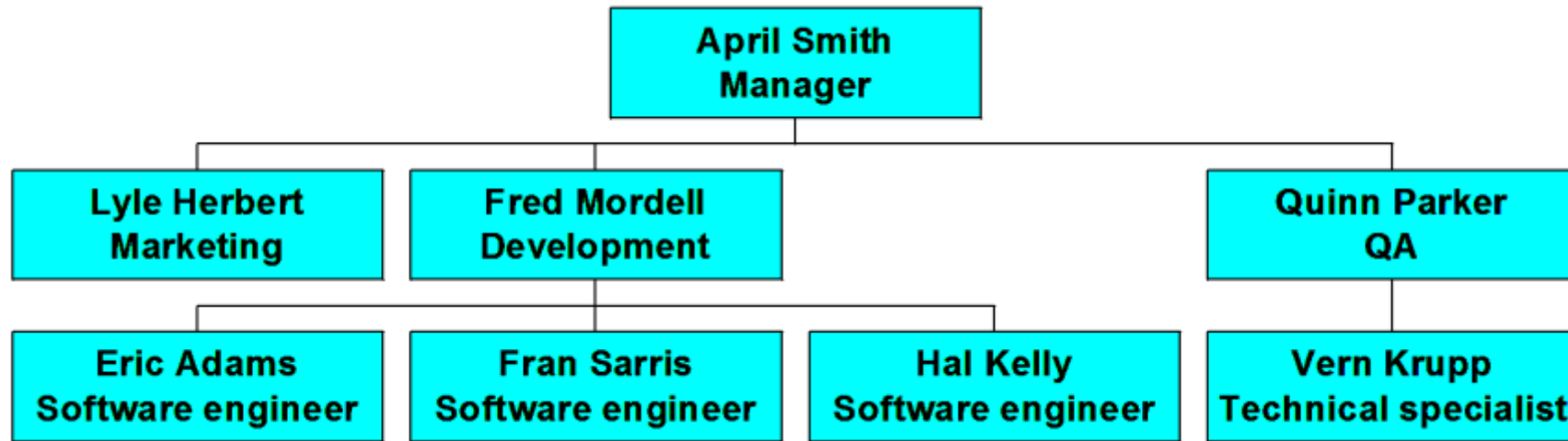
Optimal Size for Interaction (Approximate)



Optimal Size for Interaction (Approximate)



Hierarchical Project Management Organizations



Smaller Projects:

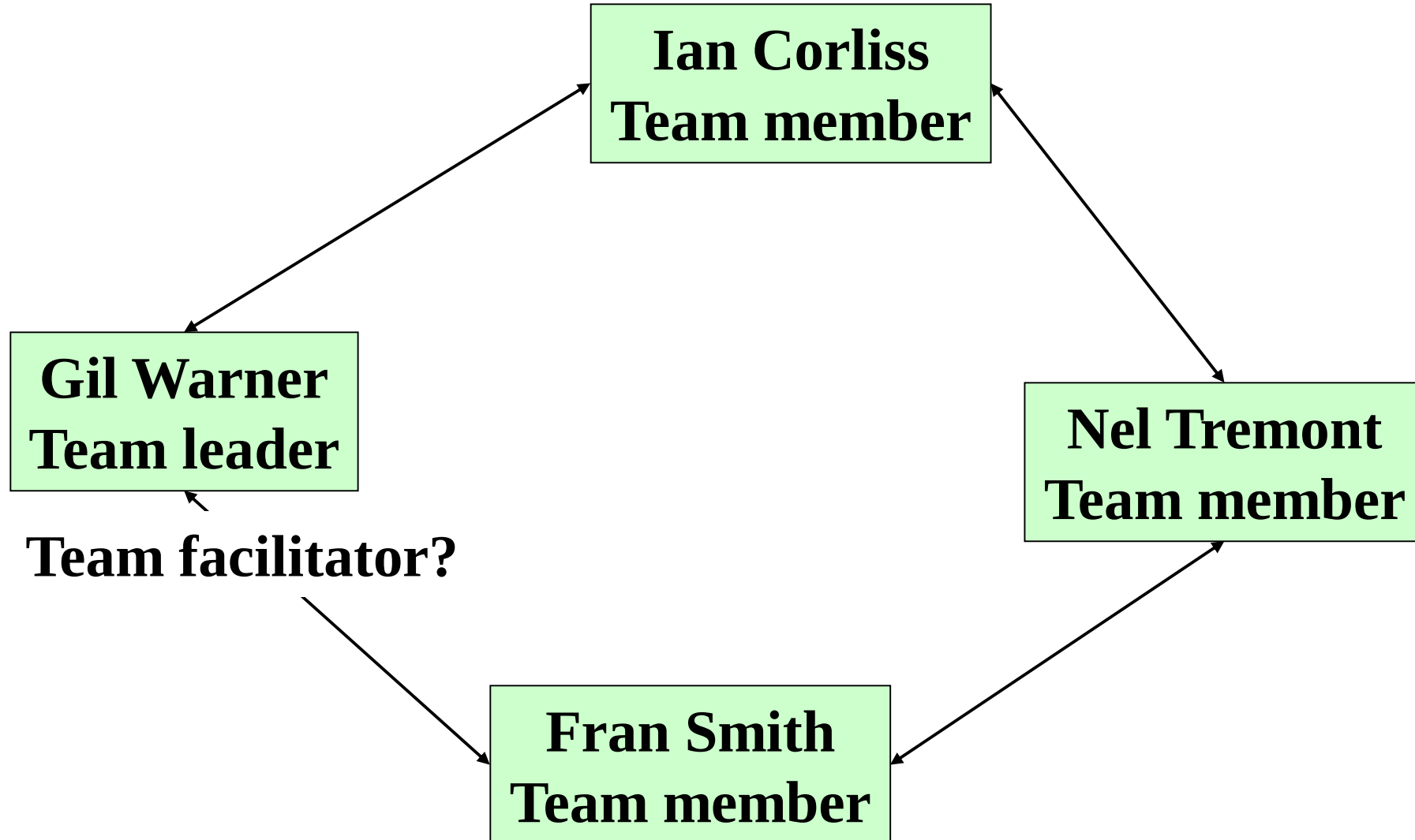
No separate Marketing?

No separate QA organization?

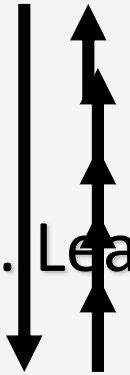
Larger Projects:

Subdivide QA into testing, ...?

**Subdivide Engineering into
system engineering, ...?**



1. Select team leader: responsibilities:
 - ensure all project aspects active
 - fill all gaps
3. Designate leader roles & document responsibilities
 - team leader: *proposes and maintains... SPMP*
 - configuration management leader: ... SCMP
 - quality assurance leader: ... SQAP, STP
 - requirements management leader: ... SRS
 - design leader: ... SDD
 - implementation leader: ... code base
2. Leaders' responsibilities:
 - propose a strawman artifact (e.g. SRS, design)
 - seek team enhancement & acceptance
 - ensure designated artifact maintained & observed
 - maintain corresponding metrics if applicable
4. Designate a backup for each leader



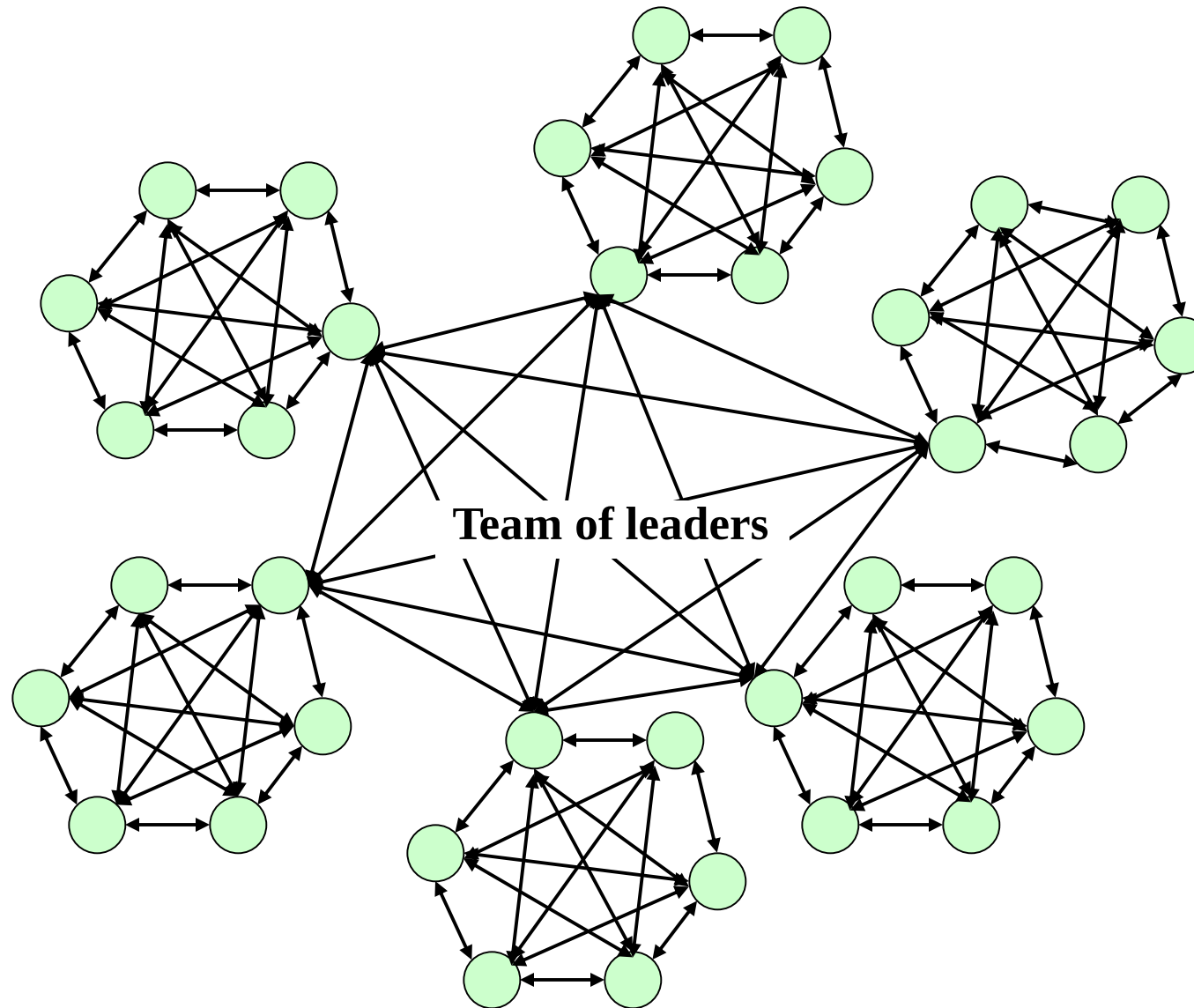


Table 2.1 Matrixed organization		Project			
		Airline reserv. project	Bank accountg. project	Molecular analysis project	Fluid mechanics project
Func- tional Unit	Project manage- ment dpt	Al Pruitt Full time	Quinn Parker Half time	Ruth Pella Full time	Fred Parsons Full time
	Marketing dpt	Oscar Mart Full time	Pete Merrill Full time	Sue More Half time	Elton Marston Full time
	Engineer- ing dpt	Hal Egberts	Ben Ehrlich	Mary Ericson	Len Engels

4. Identifying and retiring risks

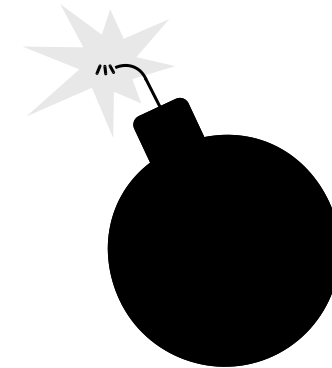
1. Identification

Mindset: *try to continually identify risks*

2. Retirement planning

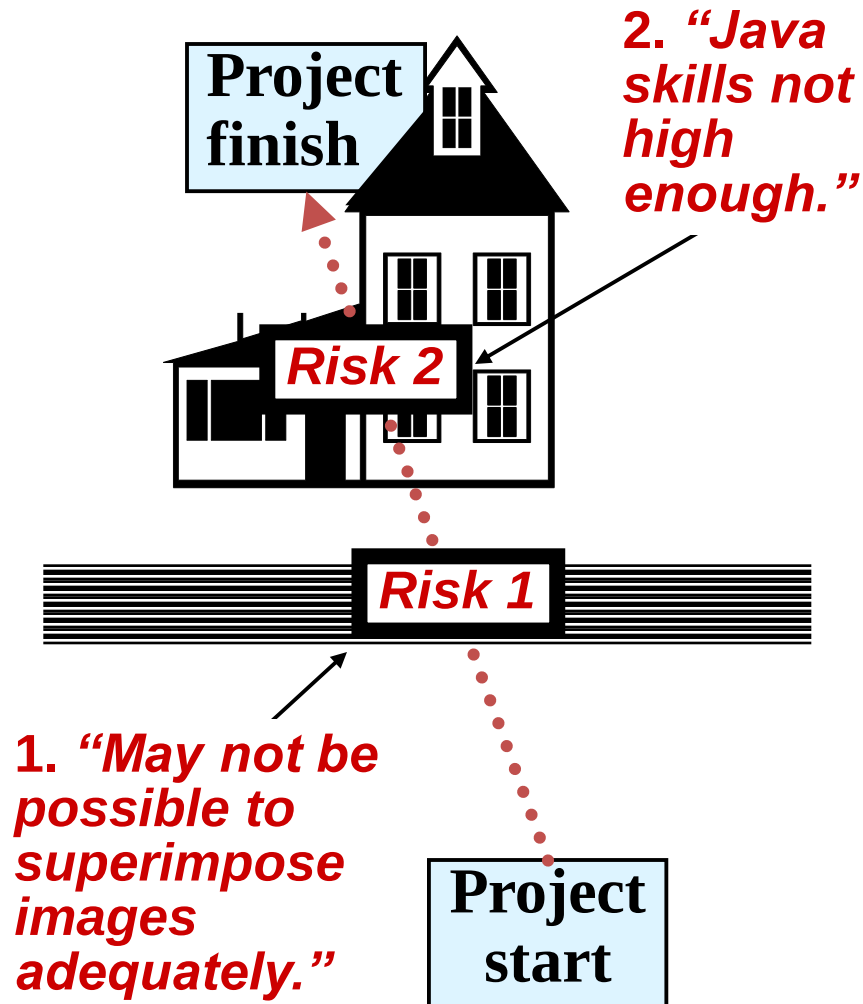
3. Prioritization

4. Retirement or mitigation



1. Lack of top management commitment
2. Failure to gain user commitment
3. Misunderstanding of requirements
4. Inadequate user involvement
5. Failure to manage end-user expectations
6. Changing scope and/or objectives
7.

Identification



Retirement

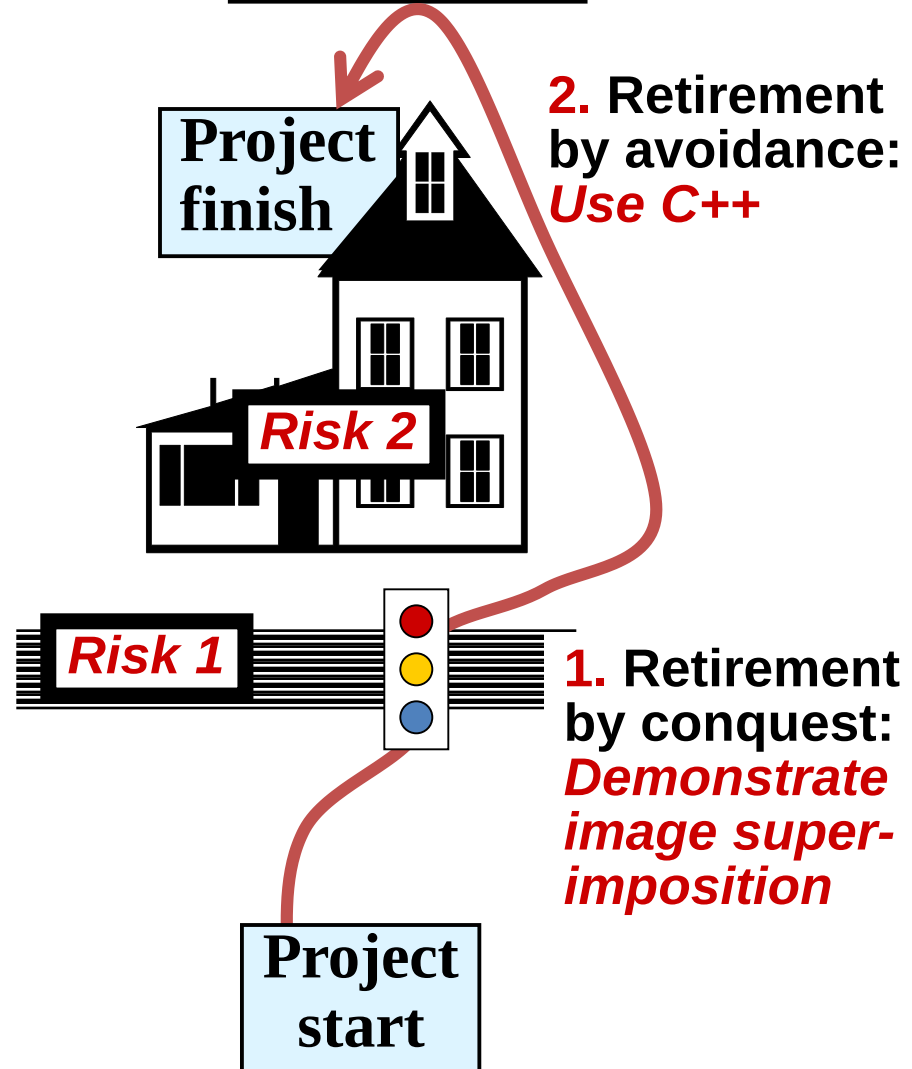


Table 2.2 A way to compute risk priorities	Likelihood 1-10 <i>1 = least likely</i>	Impact 1-10 <i>1 = least impact</i>	Retire-ment cost 1-10 <i>1 = lowest retirement cost</i>	Priority computation	Resulting priority <i>Lowest number handled first</i>
The highest priority risk	10 (most likely)	10 (most impact)	1 (lowest retirement cost)	(11-10) *(11-10) *1	1
The lowest priority risk	1 (least likely)	1 (least impact)	10 (highest retirement cost)	(11-1) *(11-1) *10	
					1000

Table 2.3 Sample Risk Analysis for *Encounter* Case Study

#	Risk title (details given above)	Like- lihood 1-10 <i>1=least likely</i>	Im- pact 1-10 <i>1=least impact</i>	Retire- ment cost 1-10 <i>1=lowest retirement cost</i>	Pri- ority <i>lowest number handled first</i>	Retirement / mitigation plan	Respon- sible engineer	Target com- pletion date
1	Super- imposing images	3	10	1	8	Experiment with Java images.	P. R.	2/1/99
2	Deficient Java skills	9	6	8	80	H.T., K.M., V.I. and L.D. to attend training course beginning 1/5/99 at Ultra Training Corp, obtain Java level 2 certification by 3/1/99 and level 3 certification by 4/15/99	H. L.	4/15/99
3	Alan Gray may be pulled off this project	3	7	9	288	Susan Ferris to inspect all of Alan's work	S.F.	Contin- ual
.	...	...	...	...	...	...	...	...

Adapted from *Software Engineering: An Object-Oriented Perspective* by Eric J. Braude (Wiley 2001), with permission.

- 1.* Each team member spends 10 mins. exploring his or her greatest fears for the project's success
- 2.* Each member specifies these risks in concrete language, weights them, writes retirement plans, (see format above) & e-mails to the team leader
- 3.* Team leader integrates and prioritizes results
- 4.^M Group spends 10 mins. seeking additional risks
- 5.^M Team spends 10 mins. finalizing the risk table
Designates responsible risk retirement engineers
- 6.** Responsible engineers do risk retirement work
- 7.^M Team reviews risks for 10 mins. at weekly meetings
responsible engineers report progress
team discusses newly perceived risks and adds them

5. Choosing development tools and support

To support project
management
schedule
work
breakdown

To support
configuration
management
For managing
requirements



For drawing designs
functional
object-oriented
use-case-based

Tracing tools
requirements to
designs
designs to code

To support testing
To support
maintenance

Example of Build vs. Buy Decision-making: Game Graphics engine

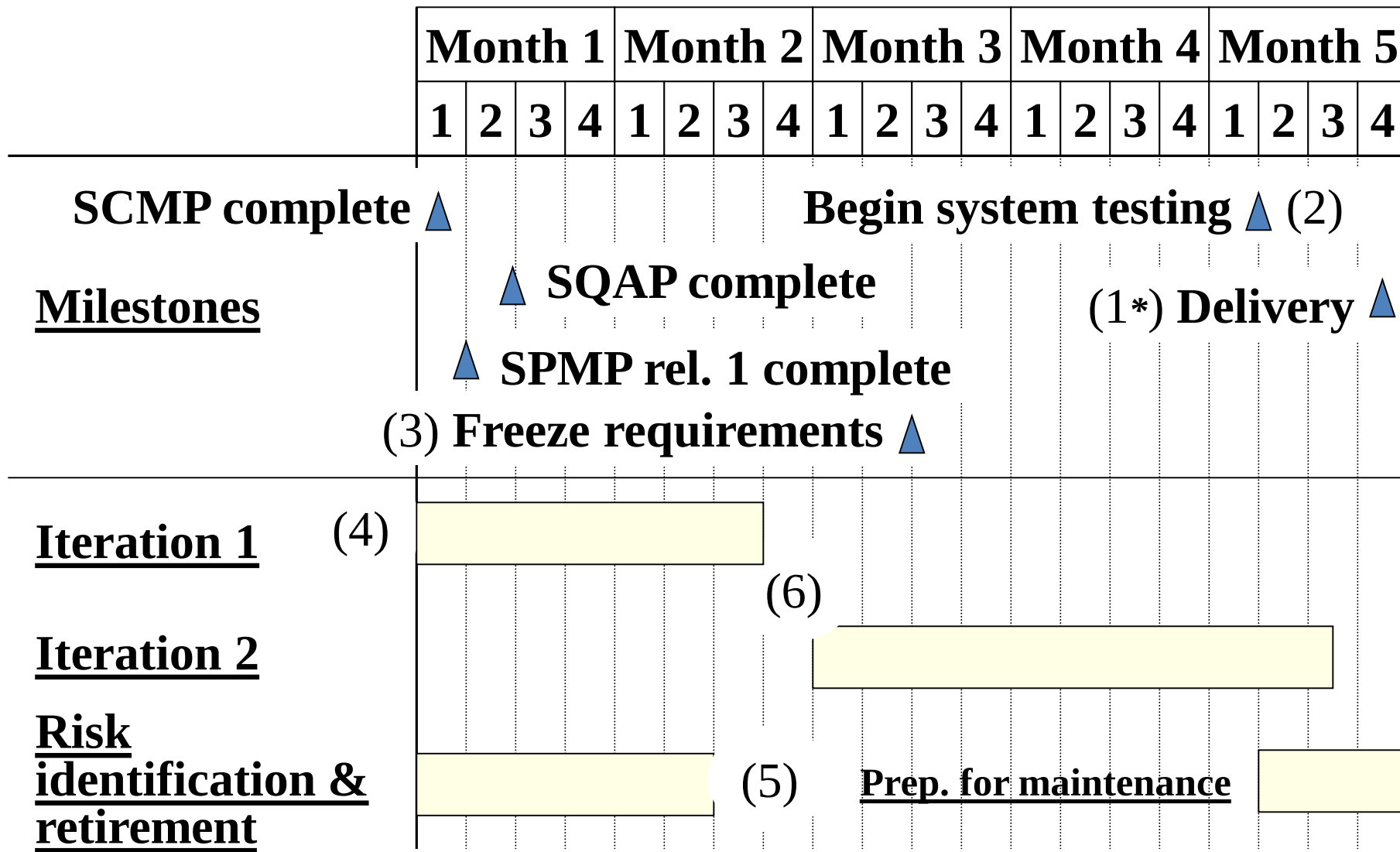
	<u>Build cost</u> (in thousands)	<u>Buy cost</u>	<u>Comments</u> multi-year costs not accounted for
Supplies	\$ 0	\$40	Purchase Ajax engine
First-person perspective	\$ 5	\$ 0	Ajax has this feature
3-D	\$10	\$ 1	Customize Ajax application
Light reflection	\$15	\$10	Customize Ajax application
TOTALS	\$30	\$51	

Build, do not buy

Table 2.4 Example of method for deciding language choice

Factor	Weight (1-10)	Benefit of Language 1 <i>1 to 10=best</i>	Benefit of Language 2 <i>1 to 10=best</i>
Internet-friendly	3	8	2
Familiarity to development team	8	3	9
Compilation speed	5	2	8
Runtime speed on processor <i>p</i>	1	7	3
<u>Score</u>		$3*8 + 8*3 +$ $5*2 + 1*7$ $= \underline{65}$	$3*2 + 8*9 +$ $5*8 + 1*3$ $= \underline{121}$

6. Creating schedules: high level planning



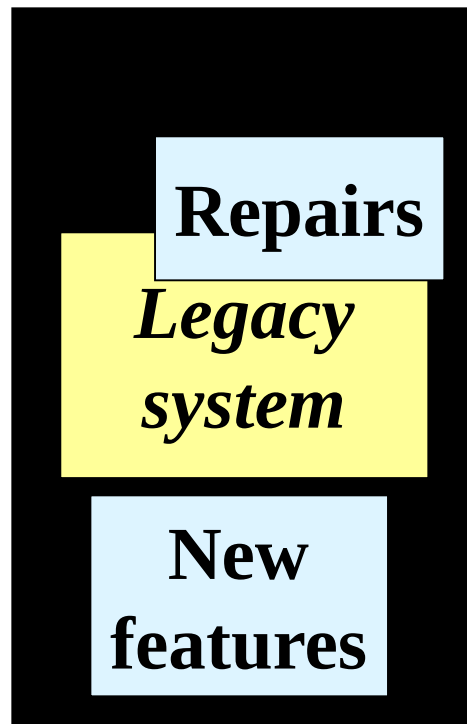
** Indicated the order in which the parts of this table were built*

1. Indicate the milestones your *must* observe
usually includes delivery date
2. Back these up to introduce the milestones you *need*
e.g., begin system testing well before delivery
3. Designate a time at which all requirements frozen
The remaining steps depend on the process used. We
will assume an iterative process.
4. Show first iteration: establishes minimal capability
usually: keep it *very modest, even trivial, in capability*
benefit: exercises the development process itself
5. Show task of identifying & retiring risks
starting from project inception
6. Show unassigned time (e.g., week) near middle?
7. Complete the schedule

	Month 1				Month 2				Month 3				Month 4				Month 5			
	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4	1	2	3	4
Milestones					Freeze requirements ▲								Complete testing ▲							
							Karen vacation						Release to production ▲							
Iteration 1	2	2	2	3	2	2	3													
													Hal vacation							
Iteration 2									4	4	4	3	3	4	4	4	4	4	4	4
Risk ID & retire	2	2	2	1	1	1		4	To be assigned											
Given team size:	4	4	4	4	4	3	3	4	4	4	4	3	3	4	4	4	4	4	4	4

7. Integrating legacy applications

Add new features (typically using same language) **or Change features** (e.g., port to new environment)



Build new application
that uses legacy system

(possibly using
different language)

